PERSONAL TECHNICAL TRAINING MANUAL

PRANA

Offline-First Edge-AI Personal Health & Environment Companion

100% On-Device | Works Offline | On-Device ML | Real SOS: SMS · Call · GPS | Flutter / Dart

APP OVERVIEW

A complete, code-grounded walkthrough — written to let you present this app to judges with confidence.

Project package: prana_health · **Version:** 1.0.0+1
Platform: Flutter 3.47.2 · Dart 3.13.2 · Android
Prepared: 13 September 2026 · for SIH judging
Basis: Direct inspection of the actual source code in this repository

Every claim in this document was checked against the real files in the codebase. Where the code does not implement something, it is marked **Not found in the current codebase** rather than guessed. Exact files are cited throughout so you can open and confirm each one yourself.

Contents

UNDERSTAND THE APP

1 Executive Summary
2 Complete Technology Stack
3 Project Architecture
4 Complete Application Flow
5 User Journey
6 Complete Feature Breakdown

HOW IT WORKS INSIDE

7 "Backend" Explanation
8 Database Deep Dive
9 ML / AI Deep Dive most important
10 AI-via-API Analysis
11 Security
12 Error Handling
13 Offline vs Online
14 Performance
15 Scalability

SHIP & DEFEND

16 Deployment Readiness
17 Deployment Architecture
18 Alternatives & Why Not
19 Technical Decision Log
20 File-by-File Codebase Map
21 Important Functions Explained
22 Data Flow
23 APIs & Third-Party Services
24 Testing
25 Current Limitations
26 Future Scope

FACE THE JUDGES

27 Judge Question Prep (40+)
28 Top 20 High-Risk Questions
29 60-Second Explanation
30 3-Minute Technical Script
31 Demo Day Flow
32 Explain Like I'm 10
33 One-Page Cheat Sheet
34 Codebase Fact Check

How to read this manual: most sections open with an **In simple terms** box (plain English), then a **Technically** box (the precise version), and an **Evidence** line pointing at the exact file. Read the plain boxes end-to-end first; drop into the technical detail only where a judge presses.

START HERE

1 Executive Summary

PRANA is an offline-first, on-device health and environment companion for a wrist-worn band. It reads your body and your surroundings, learns what "normal" looks like *for you*, flags when something is going wrong, and — if you fall or hit the SOS — can send a real SMS, place a real phone call, and share your GPS location, all without any internet connection or server.

IN SIMPLE TERMS

Imagine a small health guardian that lives entirely inside your phone. A wearable band measures your heart rate, blood-oxygen, skin temperature, motion, and the heat, humidity and air quality around you. PRANA turns those numbers into a single easy-to-read **risk score**, tells you what to do about it in plain language, and quietly learns your personal baseline over time so it can spot *your* abnormal — not a textbook average. In an emergency it can raise the alarm by itself. Nothing is uploaded anywhere.

TECHNICALLY

A Flutter/Dart (Material 3) Android application, architected offline-first with **no backend and no cloud**. A single `ChangeNotifier` controller (`PranaHealthController`) orchestrates a pipeline: BLE (or a demo generator) → sensor parsing → a frozen 16-feature vector → a **personalized on-device inference** stage (transparent physiological heuristics *plus* genuinely on-device unsupervised learning) → an uncapped 0–100 risk score → rule-based recommendations → UI, persistence, and emergency dispatch. Emergency actions are executed by the app's own native Android `MethodChannel`s (no third-party plugins).

What is genuinely notable here

- **Truly offline.** Every core feature works in airplane mode; the only outbound path is an emergency SMS/phone call over the cellular network — not the internet.
- **Real per-person learning on the device.** A running-statistics baseline (Welford + Mahalanobis distance) and a small autoencoder neural network are *trained on the phone itself* from your own calm readings — no pre-baked model, no server training.
- **Honest, transparent risk.** The domain risks (heat, breathing, cardiovascular, fatigue) come from hand-tuned physiological formulas you can read and defend line-by-line — not an opaque black box.
- **Emergency response that actually fires.** On a detected fall (after a cancelable countdown) or a manual SOS, the app dispatches a real SMS with identity + vitals + a Google-Maps location link, and places a real call, through native Android APIs.

BE HONEST ABOUT THIS ON STAGE

The shipped TensorFlow-Lite model files are **not** used at runtime (the live path is the personalized service described above). There is **no encryption at rest** yet (the encrypted-database stack is scaffolded but inactive). There are **no accuracy/precision/recall numbers**, and the displayed model confidence is a fixed placeholder. The release build is currently **debug-signed**. Section 25 lists every such limitation, and Section 33 gives you the exact "things I must not claim" list. Owning these honestly will impress judges more than hiding them.

2 WHAT IT'S BUILT WITH

Complete Technology Stack

IN SIMPLE TERMS

The whole app is built in **Flutter** (Google's toolkit for building phone apps), written in the **Dart** language. It stores data in two simple on-device stores, talks to the wearable over **Bluetooth**, and reaches Android's phone features (SMS, calling, GPS, file sharing) through a thin bit of native **Kotlin** code the team wrote themselves.

Core platform

Layer	Technology	Notes (verified from the repo)
Language	Dart 3.13.2	SDK constraint `^3.13.2` in `pubspec.yaml`
Framework	Flutter 3.47.2 (stable)	UI toolkit; single codebase
Design system	Material 3, **light theme only**	`app_theme.dart` — `useMaterial3:true`; no dark theme defined
State management	Provider + one `ChangeNotifier`	Single controller as composition root (see §3)
Target OS	Android	`compileSdk 37`, Java/Kotlin 17 (`build.gradle.kts`)
Native glue	Kotlin, plain Android SDK	Two own `MethodChannels`; no 3rd-party native plugins

Dependencies actually declared (pubspec.yaml → resolved in pubspec.lock)

Being able to say which dependencies are *live* versus *declared-but-inert* is exactly the kind of precision judges reward. This table is the ground truth.

Package · version	Purpose	Actually used at runtime?
flutter_blue_plus 1.36.8	Bluetooth LE to the band	**Yes** — `bluetooth_sensor_data_source.dart`
provider 6.1.5+1	State management	**Yes** — main.dart + ~17 screens
shared_preferences 2.5.5	Key-value storage	**Yes** — the app's real settings/profile/summary store
sembast 3.8.10	Pure-Dart NoSQL DB	**Yes** — backs the ML pipeline (`vitals_sample_store.dart`)
pdf 3.13.0	PDF generation	**Yes** — doctor report (`report_pdf_builder.dart`)
path_provider 2.1.6 · path 1.9.1	File-system dirs / paths	**Yes** — report, export, sembast DB
intl 0.19.0	Date/number formatting	**Yes** — throughout the UI
tflite_flutter 0.12.1	On-device TFLite inference	**No (dead code)** — imported only by an un-instantiated service
flutter_secure_storage 11.0.0	Keystore for a DB passphrase	**No** — reachable only via a never-called key manager
drift 2.34.4 · sqlite3 3.5.2	SQLite ORM + bindings	**No** — no `package:drift` import anywhere
sqlite3_flutter_libs 0.6.0 · sqlcipher_flutter_libs 0.7.0	Native SQLite / SQLCipher libs	**No** — companions to the unused SQL stack
cupertino_icons 1.0.9	iOS-style icons	**No** — template boilerplate

Dev dependencies: `flutter_test` (used — 6 test files), `flutter_lints 6.0.0` (used via `analysis_options.yaml`), plus `drift_dev 2.34.6`, `build_runner 2.16.1` and `test 1.31.1` which are unused/redundant (no generated files exist).

EVIDENCE

`pubspec.yaml, pubspec.lock, analysis_options.yaml, android/app/build.gradle.kts`

WHY THE INERT PACKAGES MATTER

An entire **encrypted-SQLite persistence stack was scaffolded but never wired in** (drift, sqlite3, sqlcipher, flutter_secure_storage). The app actually persists through SharedPreferences and sembast. This is the single most important "gotcha" to pre-empt: if a judge greps the pubspec and asks "so you're using an encrypted SQLCipher database?", the correct, confident answer is "those are scaffolded for a future encrypted store; today persistence is SharedPreferences + sembast, in plaintext — here's the file that proves it." (See §8, §11, §25.)

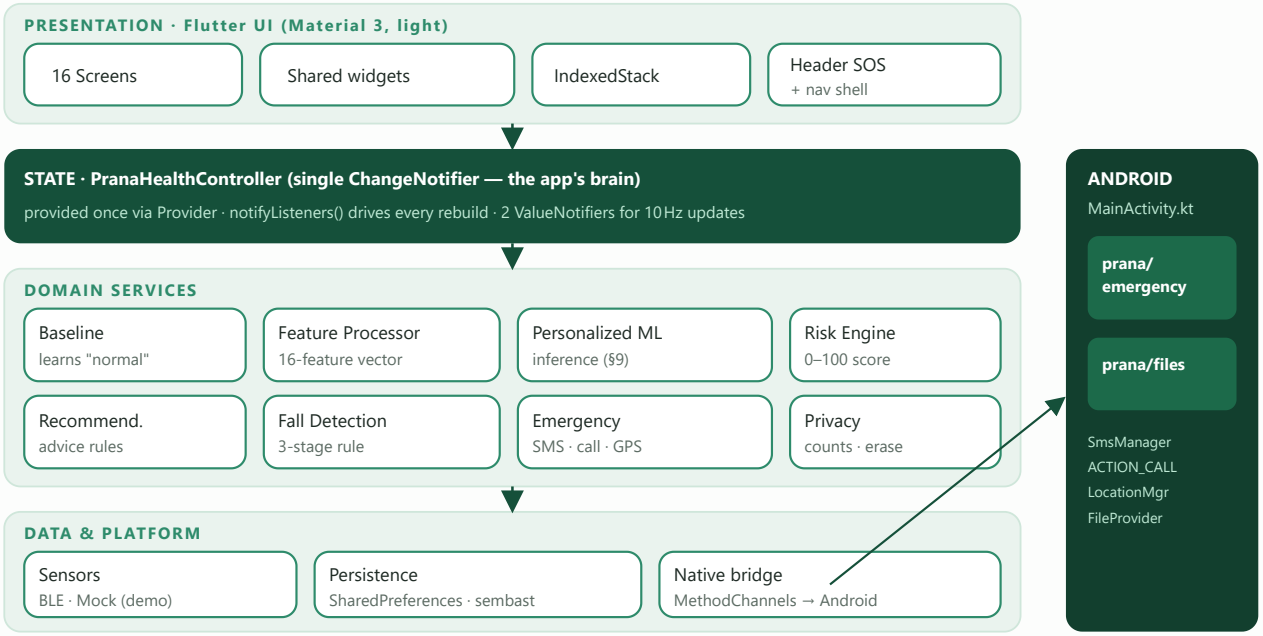
3 HOW THE PIECES FIT
Project Architecture

IN SIMPLE TERMS

Think of the app as four stacked layers. At the top is what you **see** (the screens). Under that sits one **brain** object that holds all the current numbers and tells the screens when to refresh. The brain calls a set of **specialist helpers** (one learns your baseline, one scores risk, one writes advice, one watches for falls, one handles emergencies). At the bottom sits where the **data comes from and goes** (the Bluetooth band or a demo generator, two small on-device stores, and a thin native bridge to Android).

TECHNICALLY

A layered, single-controller architecture. `PranaHealthController` (a `ChangeNotifier`) is the composition root and single source of truth; it owns every service and is provided once at the top of the widget tree. Screens subscribe with `context.watch` / `context.read`. Two auxiliary providers expose the raw vitals store (read-only, for CSV export) and a live ML-personalization status notifier. Two `ValueNotifiers` inside the controller (a 10 Hz motion trace and the ML status) push high-frequency updates without rebuilding the whole tree.



Layered architecture. One controller mediates between the UI and every service; the native Kotlin bridge is reached only through the data layer. Verified against `main.dart`, `app.dart`, `prana_health_controller.dart`, `MainActivity.kt`.

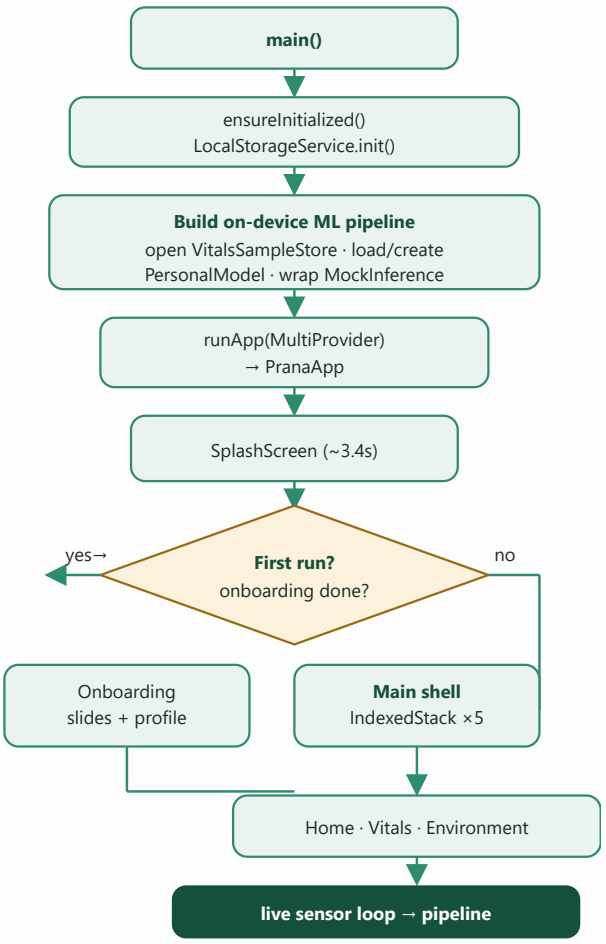
EVIDENCE

`lib/main.dart:13-63` · `lib/app/app.dart` · `lib/core/services/prana_health_controller.dart:38-46` · `android/.../MainActivity.kt`

4 FROM TAP TO INSIGHT
Complete Application Flow

IN SIMPLE TERMS

When you open the app it sets up its storage and its learning model, shows a short animated splash, then either walks a first-time user through onboarding or drops a returning user straight onto the home screen. From then on, every couple of seconds a fresh reading arrives, the brain runs it through the pipeline, and the screens update.



Startup & branch. `main.dart:13–63, app.dart:42–55.`

The per-reading pipeline

Once the shell is up, a sensor stream (real BLE, or the demo generator in Demo mode) delivers a reading roughly every 2 seconds. For each reading the controller runs:

1. **Parse** the raw packet into typed readings (health, motion, environment).
2. **Update baseline** — feed calm readings into the running "normal" model.
3. **Extract features** — build the frozen 16-number feature vector.
4. **Infer** — run the personalized inference stage → per-domain risks + an overall anomaly signal.
5. **Score risk** — combine into an uncapped 0–100 risk value + level.
6. **Recommend** — pick specific advice via the rule cascade.
7. **Notify** — call `notifyListeners()` so subscribed screens repaint.
8. **Persist** — roll the day's numbers into a `DailySummary`; store calm samples for learning.
9. **Watch for falls** — the motion stream feeds fall detection independently; a fall arms the emergency countdown.

IMPORTANT DEFAULT

On a **fresh install the sensor source is real BLE** (demo mode defaults off). So with no band paired and Demo mode not enabled, the screens legitimately show "no live data yet." For a demo, either pair a band or turn on Demo mode first — otherwise the home screen is intentionally blank.

EVIDENCE

`lib/main.dart · lib/app/app.dart · lib/core/services/prana_health_controller.dart (reading pipeline; notifyListeners at ~:458)`

5 WHAT A PERSON DOES
User Journey

IN SIMPLE TERMS

A first-time user is shown a few explainer screens and asked for the basics that make an emergency alert useful (name, age, a condition or two, an emergency contact). After that, they live mostly on the Home screen watching their risk score, dip into Vitals or Environment for detail, and — only if something goes wrong — hit SOS or let a detected fall raise the alarm.

Stage	What the user sees / does	Where it lives

First launch	Animated splash, then 3 explainer slides + a real profile-setup step (so the first SOS names a real person, not the sample profile)	`splash_screen.dart`, `onboarding_screen.dart`
Home	The big risk score, live vitals overview, and the single most relevant piece of advice	`dashboard_screen.dart`
Vitals	Per-metric dials & charts (HR, SpO_2, temperature, sleep) read against the learned baseline	`health_screen.dart`
Environment	Feels-like heat, temperature, humidity, air quality (AQI), safe-exposure time, with per-condition advice	`environment_screen.dart`
Insights	The on-device analysis: the risk score explained, with the specific drivers behind it	`insights_screen.dart`
Me / Profile	Profile, learned baseline, emergency contacts, settings — all editable and persisted	`profile_screen.dart`
Emergency	Manual SOS (swipe to confirm); on a fall, a 10-second cancelable countdown then auto-dispatch	`emergency_screen.dart`, `sos_action_button.dart`
Extras	History (day-by-day), doctor PDF report, CSV data export, device/BLE controls, privacy counts, demo lab	history / report / data_export / devices / privacy / demo

TECHNICALLY

Navigation is hybrid: the five primary destinations live in an `IndexedStack` inside `MainNavigationShell` (preserving scroll state), while secondary screens (emergency, devices, privacy, demo lab, fall detection, history, report) are pushed as named `MaterialPageRoutes`. A persistent SOS disc sits in every destination's header via a small `EmergencyLauncher InheritedWidget`; the emergency screen is also auto-surfaced by a stream subscription so a fall opens it even if the user is elsewhere.

EVIDENCE

`lib/app/app.dart:76-513 · lib/app/routes.dart:19-38 · lib/features/emergency/sos_action_button.dart`

6 EVERYTHING IT DOES
Complete Feature Breakdown

Feature	What it does	Real / Heuristic / Demo
Live vitals monitoring	HR, SpO_2, skin temperature, motion streamed from the band over BLE and parsed into typed readings	**Real** (needs a band)
Environment monitoring	Ambient temperature, humidity, AQI from the band; heat-index computed on the phone	**Real** (from band)

Personal baseline learning	Learns your resting normal from calm readings; shows deviations against it	**Real** on-device
Risk score (0–100)	Single combined score + level, uncapped natural range, with named drivers	**Heuristic** + personalization
Per-domain risks	Heat stress, respiratory, cardiovascular, fatigue sub-scores	**Heuristic** (physiological formulas)
Personalized anomaly	Flags "unusual for *you*" via running stats + a small on-device autoencoder	**Real** on-device ML
Recommendations	Specific, situation-aware advice (not "drink water" clichés)	**Rule-based**
Fall detection	3-stage rule on the band's IMU → arms emergency countdown	**Real** (band IMU)
Emergency dispatch	Real SMS (identity + vitals + Maps link) + real phone call + GPS	**Real** (native Android)
Doctor report (PDF)	Multi-page A4 PDF with charts, baseline, period summary, disclaimer	**Real** (pure-Dart)
Data export (CSV)	Exports recorded calm ML readings incl. the exact feature vector	**Real**
History	Day-by-day record from persisted daily summaries (≤90 days)	**Real**
Privacy panel	Real counts of what's stored locally; export / erase	**Real**
Disaster scenarios	6 replayable scenarios (heatwave, air pollution, flood, cyclone, elderly, normal)	**Demo only**
Sleep & recovery	Sleep staging panel	**Demo** (fixed sample data)

EVIDENCE

`lib/features/* (per-screen), mock_sensor_data_source.dart, fall_detection_service.dart, emergency_dispatcher.dart, report_pdf_builder.dart, data_export_service.dart`

7 WHERE'S THE SERVER?
"Backend" Explanation

THERE IS NO BACKEND — AND THAT'S THE POINT

PRANA has **no server, no cloud, no REST/GraphQL API, no database server, and no authentication service.** Searched for and **not found in the current codebase.** This is a deliberate offline-first design, not a missing piece.

IN SIMPLE TERMS

Everything a "backend" would normally do — store data, run the analysis, make decisions — happens right on the phone. The phone *is* the backend. That's why it works with no signal, in a flood or a remote village, and why your health data never leaves the device.

TECHNICALLY

What would be "backend responsibilities" are implemented as an on-device service layer coordinated by `PranaHealthController`: ingestion (sensor sources), transformation (feature processor), inference (personalized ML service), decision (risk + recommendation engines), persistence (SharedPreferences + sembast), and side-effects (native emergency/file channels). There is no network client in the app at all for its own data.

The only "API-like" table there is (on-device method channels)

Channel	Methods	What it reaches
`prana/emergency`	requestPermissions, permissions, getLocation, sendSms, placeCall	Android SmsManager, ACTION_CALL, LocationManager
`prana/files`	shareFile, openFile	androidx FileProvider (share/open the PDF/CSV)

EVIDENCE

`android/app/src/main/kotlin/.../MainActivity.kt (EMERGENCY_CHANNEL, FILES_CHANNEL) · emergency_dispatcher.dart · report_service.dart (FileSharer)`

8 WHERE DATA LIVES
Database Deep Dive

IN SIMPLE TERMS

There are two little storage boxes on the phone. The first (SharedPreferences) is a simple labelled key-value box — it holds your profile, your learned baseline, your settings, and up to 90 daily summaries. The second (sembast) is a tiny on-device database that holds the raw "calm" readings the learning model trains on, plus the model itself. Both are plain and local; nothing is encrypted yet.

① SharedPreferences

Via `local_storage_service.dart`. Simple key→value.

- User profile (name, age, sex, conditions, contact)
- Learned baseline (normal HR/SpO_2/temp, readings learned)
- App settings + onboarding-complete flag
- **DailySummary × up to 90** (retention 90 days)

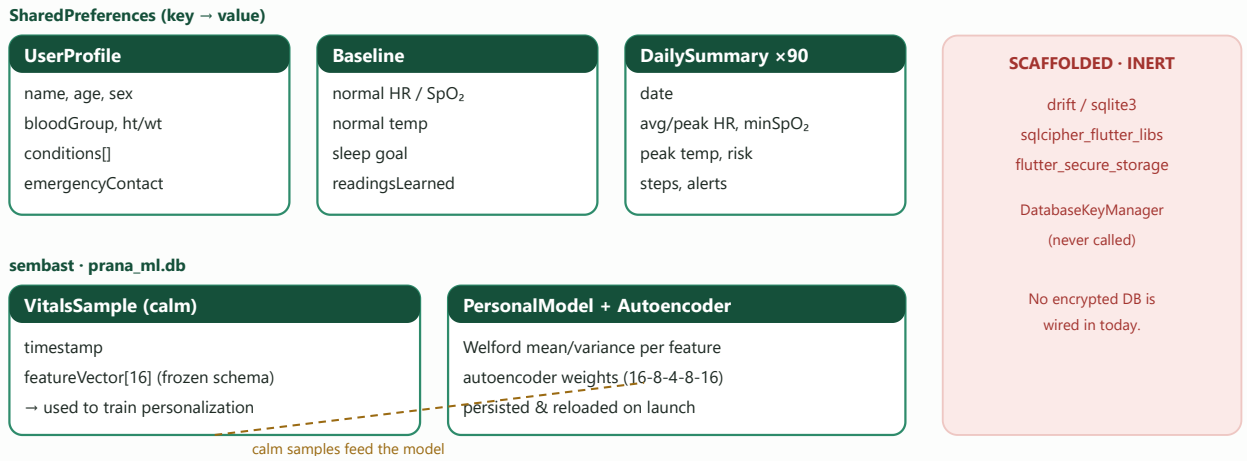
No raw minute-by-minute history is kept here — only daily rollups.

② sembast (prana_ml.db)

Via `vitals_sample_store.dart`. Pure-Dart NoSQL.

- Recorded **calm feature vectors** (16 numbers + timestamp) — the ML training data
- The persisted **PersonalModel** (running stats) & autoencoder weights

Read-only copy exposed to the CSV export screen via a Provider.



Two live stores (green) plus the scaffolded-but-inactive encrypted stack (red). There is no relational schema; both stores are document/key-value.

SAY THIS, NOT THAT

✓ "Data is stored locally in SharedPreferences and a sembast NoSQL database." ✗ "We use an encrypted SQLCipher database." (The SQLCipher stack is present in `pubspec.yaml` but **not wired in**; storage is currently plaintext — see §11 & §25.)

EVIDENCE

`local_storage_service.dart` · `vitals_sample_store.dart` · `database_key_manager.dart (orphaned)` · `app_constants.dart (dailySummaryRetentionDays=90)`

9 MOST IMPORTANT SECTION
ML / AI Deep Dive

This is the section judges probe hardest, so it is the section you must know cold. The golden rule: describe each layer as exactly what it is — real learning where it is real, transparent heuristics where they are heuristics, and dormant where a file ships but never runs.

The four kinds of "AI" in this app — kept honestly separate

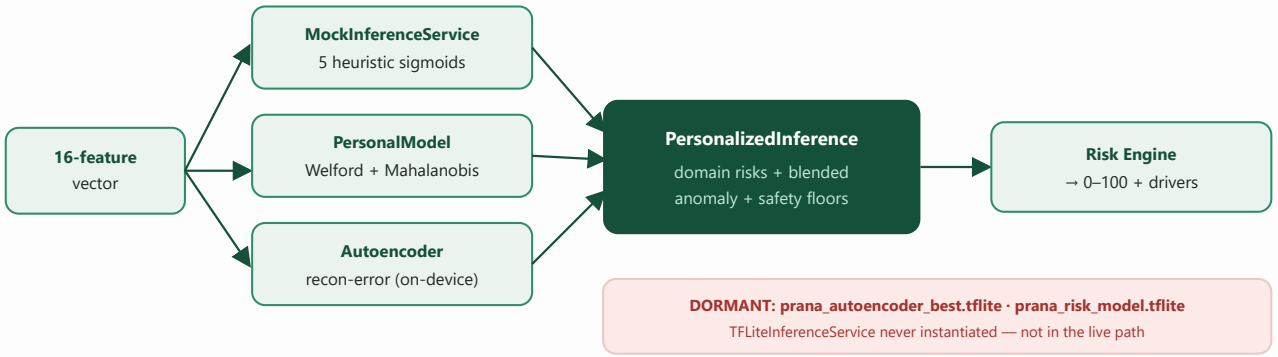
Category	What it is here	Where
Real, on-device ML ✓	(a) A per-person **running-statistics baseline** (Welford mean/variance) with a **Mahalanobis distance** anomaly score. (b) A small **autoencoder neural network** (16-8-4-8-16) **trained on the device itself** by SGD/back-prop in a background isolate; reconstruction error = "unusual for you."	`personal_model.dart`, `autoencoder.dart`

Rule-based / heuristic ◐	Five hand-tuned logistic-style formulas (sigmoids) for heat-stress, respiratory, cardiovascular, fatigue and overall risk over the 16-feature vector. Coefficients come from **published physiological thresholds**, **not** fitted to a dataset.	`mock_inference_service.dart`
API-based AI ✗	**None.** No OpenAI / Gemini / cloud model. Not found in the current codebase.	—
Mock / dormant ✗	Two `.tflite` files ship. One (`prana_autoencoder_best.tflite`, ~7.4 KB) is a genuine offline-trained model but is **never loaded**. The other (`prana_risk_model.tflite`, 92 bytes) is placeholder text. The TFLite service that would load them is **never instantiated**.	`assets/models/*.tflite`, `tflite_inference_service.dart` (dead)

What actually runs at inference time (the live path)

TECHNICALLY — THE WIRING, VERIFIED

At startup, `main.dart` constructs a `PersonalizedInferenceService` that *wraps* a `MockInferenceService`, and injects it into the controller via the `customMlService` parameter. The controller uses `customMlService ?? MockInferenceService()` — so with the injection present, **PersonalizedInferenceService is the live inference engine.** For each 16-feature vector it produces: (1) the five domain probabilities straight from the heuristic `MockInferenceService` (unchanged, transparent), and (2) a **personalized overall-anomaly** signal that blends the Mahalanobis distance from the `PersonalModel` and the autoencoder's reconstruction error, with hard physiological floors so a genuinely dangerous reading can't be "learned away" as normal.



Live inference (green) vs. shipped-but-unused TFLite (red). Verified in `main.dart:30–42`, `personalized_inference_service.dart`, `prana_health_controller.dart:170–174`.

The 16-feature vector (frozen schema v1.1.0)

Every model consumes the same fixed vector, defined once in `feature_schema.dart` and computed in `feature_processor.dart`. Freezing the schema means an exported CSV can be de-normalized back to real units exactly — the export is self-verifying. It carries absolute vitals (HR, SpO_2, temp), baseline-relative deviations (e.g. HR % deviation), an HR

trend (an OLS slope from `hr_trend_calculator.dart`), environmental terms (heat index, AQI, humidity) and a categorical activity level.

THE THREE THINGS YOU MUST NOT CLAIM ABOUT THE ML

✗ "It uses a trained neural network for the risk score." → The *risk* comes from heuristics; the trained autoencoder contributes only a personalized *anomaly* signal, and the shipped `.tflite` is unused. ✗ "It's 90-something-percent accurate." → There are **no** accuracy/precision/recall metrics; the on-screen confidence (~0.92) is a **fixed placeholder**. ✗ "It diagnoses conditions." → It is a wellness/early-warning aid, explicitly not a diagnostic device (the PDF report says so).

THE ONE-PARAGRAPH HONEST ML PITCH (MEMORIZE THIS)

"PRANA does real machine learning *on the device*: it learns each person's normal with running statistics and a small autoencoder trained on the phone itself, so it can flag what's unusual *for you* — completely offline. On top of that, the specific risk scores come from transparent, physiology-based formulas we can explain line by line, rather than an opaque black box. We also ship a separately-trained TensorFlow-Lite model, but I'll be upfront: it isn't wired into the live path yet, and we don't claim accuracy numbers we haven't measured."

EVIDENCE

`feature_schema.dart · feature_processor.dart · mock_inference_service.dart:51–96 · personalized_inference_service.dart:71–139 · personal_model.dart:41–68 · autoencoder.dart:18–405 · tflite_inference_service.dart (dead)`

10 IF AI CAME FROM A SERVICE
AI-via-API Analysis

NOT FOUND IN THE CURRENT CODEBASE

PRANA calls **no external AI or ML API**. There is no OpenAI, Anthropic, Google Gemini, Hugging Face, or any cloud-inference client; no API keys, no model endpoints, no HTTP calls for inference. All intelligence runs on-device (see §9).

WHY THIS IS A STRENGTH, NOT A GAP

Because nothing is sent to a third-party AI service, PRANA keeps working with no internet, incurs no per-request cost, adds no network latency, and never exposes health data to an outside provider. For a disaster/rural-resilience product, "no API dependency" is a feature to lead with, not apologise for.

IF YOU WERE ASKED TO ADD ONE LATER

The clean insertion point is the `MlInferenceService` abstraction that `PersonalizedInferenceService` and `MockInferenceService` already implement — a future cloud service could implement the same interface and be swapped in behind a connectivity check, without touching the controller. Today, that is design headroom, not a shipped feature.

11 HOW SAFE IS THE DATA

Security

IN SIMPLE TERMS

The strongest privacy protection PRANA has is that data **never leaves the phone** — there's no account, no server, nothing to breach remotely. The weaker spot is that the data saved on the phone is **not encrypted** yet, so anyone with full access to an unlocked device could read the app's stored files.

Aspect	Status	Detail (verified)
Data leaves the device?	**Never***	No network client for app data. *Only an emergency SMS/call goes out, over cellular.
Encryption at rest	**None**	SharedPreferences + sembast are plaintext; SQLCipher stack scaffolded but inert
Encryption in transit	**N/A**	Nothing transmitted (SMS is inherently unencrypted by design of SMS)
User authentication	**N/A**	Single-user, on-device; no login by design
Permissions requested	**Scoped**	BLE, location (needed for BLE scan), SEND_SMS, CALL_PHONE — each tied to a real feature
Secrets / API keys	**None**	No secrets in the repo (nothing to leak)
Release signing	**Debug key**	Release build uses the debug keystore; no production key configured
User data control	**Yes**	Privacy screen shows real counts and can export/erase local data

HONEST FRAMING FOR JUDGES

"Our threat model is 'a lost or shared phone.' We win on the network dimension — there is literally no server to attack and no data in transit. We're candid that encryption-at-rest isn't wired in yet: the SQLCipher + secure-storage scaffolding is in place, and turning it on is the top security item on our roadmap (§26)."

EVIDENCE

`AndroidManifest.xml (permissions) · build.gradle.kts:37–41 (debug signing) · database_key_manager.dart (inert) · privacy_service.dart`

12 WHEN THINGS GO WRONG

Error Handling

IN SIMPLE TERMS

The app is built to **degrade gracefully**: if a sensor packet is garbled it's ignored, if GPS isn't available it says so instead of faking a location, and if a permission is denied the emergency flow falls back to copying the alert text so a human can still send it.

Failure	How it's handled (verified)
Malformed sensor packet	`SensorParser` returns `null` on invalid JSON (unit-tested); the reading is skipped, no crash
Missing sensor fields	Parser only emits streams for keys actually present — no invented default values
No GPS fix	Emergency reports "Location unavailable" rather than a fake coordinate (unit-tested)
Emergency permission denied / non-Android	Native dispatch degrades to clipboard copy of the alert; the flow still completes
PDF/CSV share fails	`FileSharer` returns `false` instead of throwing; UI can inform the user
No saved ML model on first run	Falls back to `PersonalModel.fresh()` and starts learning from scratch
No band / no live data	Screens render explicit empty-states ("No conditions yet", "Connect device") rather than blank crashes

CANDID GAP

Error handling is feature-local (guards, null-returns, fallbacks) rather than a global crash-reporting/telemetry layer — there is no Sentry/Crashlytics-style reporter (by design, since the app is offline). Worth stating plainly if asked.

EVIDENCE

`sensor_parser.dart (+ test) · emergency_service.dart (+ test) · report_service.dart (FileSharer) · main.dart (PersonalModel.fresh fallback)`

13 WHAT NEEDS A SIGNAL
Offline vs Online

IN SIMPLE TERMS

Basically everything works offline. The only thing that touches the outside world is an emergency alert, and even that uses the phone network (SMS/call), not the internet.

Capability	Needs internet?	Path
Vitals & environment monitoring	**No**	Bluetooth to the band
Risk scoring & recommendations	**No**	On-device compute
ML personalization & training	**No**	On-device (background isolate)
Storage / history / privacy	**No**	SharedPreferences + sembast
PDF report & CSV export	**No**	Generated locally; shared via file provider
Emergency SMS + call	**No internet**	Cellular network (SMS/telephony)
Any cloud sync / accounts	—	Not implemented (none exists)

EVIDENCE

`Whole codebase: no HTTP/Dio/network client for app data; emergency_dispatcher.dart uses the native channel only`

14 DOES IT STAY SMOOTH
Performance

IN SIMPLE TERMS

The heavy work (learning the model) is pushed off the screen's thread so the UI never stutters, screens keep their state when you switch tabs, and the app only keeps a short rolling window of recent points in memory rather than everything.

- **Isolate for training.** Autoencoder training runs in a Flutter `compute()` isolate — off the UI thread, so scrolling stays smooth during learning.
- **Targeted repaints.** Two `ValueNotifiers` (the 10 Hz motion trace and ML status) update only their widgets via `ValueListenableBuilder`, avoiding full-tree rebuilds.
- **State preservation.** The five main tabs live in an `IndexedStack`, so switching tabs doesn't rebuild or lose scroll position.
- **Bounded memory.** In-memory history buffers are capped (roughly 50–150 recent points); only daily rollups persist long-term.
- **Light inference.** The live inference is small-matrix arithmetic + a tiny neural net — cheap enough to run every ~2 s tick.

NOT MEASURED

There are **no formal benchmarks** (frame timings, memory profiles, battery draw) in the repo. Claims above are architectural, not measured — say "designed for" rather than citing numbers you don't have.

EVIDENCE

`autoencoder.dart (compute isolate) · prana_health_controller.dart (motionTick ValueNotifier ~:105) · app.dart (IndexedStack)`

15 CAN IT GROW
Scalability

IN SIMPLE TERMS

Because there's no server, PRANA "scales" trivially in one sense — every phone is fully self-contained, so a million users cost you no servers. What it does **not** do today is combine data across users or give an authority a fleet-wide view.

Scales naturally

- Zero server cost — compute is on each device
- No shared bottleneck, no rate limits
- Works the same for 1 or 1,000,000 users
- Privacy scales with it (nothing centralized)

Doesn't scale (by design, today)

- No cross-user aggregation / population dashboards
- No multi-device sync per user
- No central fleet management for authorities
- Model learns per-device only (no federated pooling)

GROWTH PATH

Adding an *optional* sync/aggregation tier later doesn't require re-architecting: the controller already produces clean `DailySummary` objects and the export layer already serializes readings — those are natural upload units for a future opt-in backend (§26).

16 READY TO SHIP?
Deployment Readiness

IN SIMPLE TERMS

As a working prototype for judging, it's in good shape: it builds, runs on a real phone, and the headline features work. As a store-ready product, a few production chores remain — mainly proper app signing, a real app ID, encryption, and a background service for locked-screen reliability.

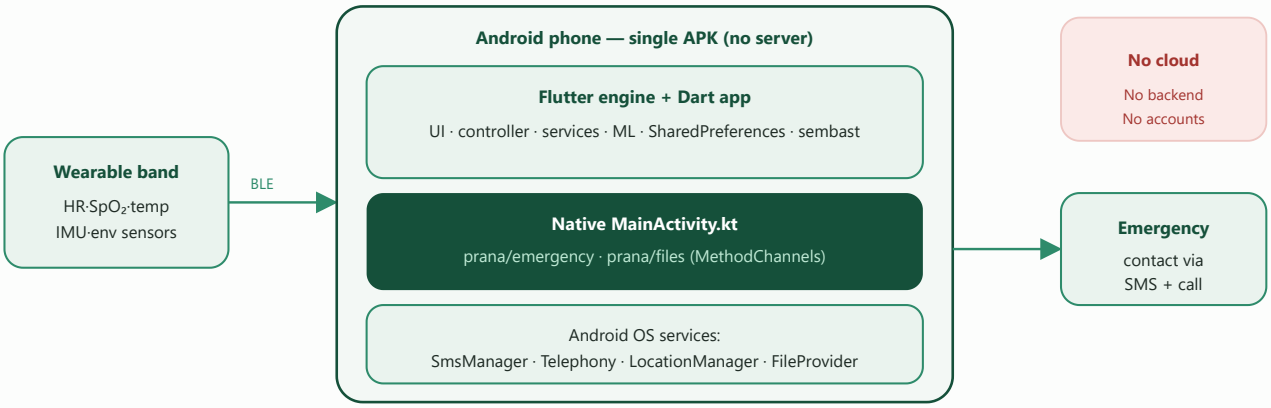
	Item	Status
✅	Compiles & runs on a real Android device	Analyzer clean; runs on-device
✅	Core features functional	Vitals, risk, recommendations, history, report, export
✅	Real emergency dispatch	Native SMS + call + GPS via `prana/emergency`
✅	On-device ML personalization	Baseline + autoencoder train/persist on device
✅	Unit tests for core logic	6 files, ~18 cases (§24)
⚠️	Release signing	Debug-signed; no production keystore/`key.properties`
⚠️	Application ID	Still `com.example.sih_test_app_new` (template default; in-file TODO)
⚠️	Locked-screen fall reliability	No foreground/background service declared
❌	Encryption at rest	Scaffolded, not wired in
❌	Widget / integration tests	Only unit tests exist
❌	iOS native emergency/file channels	Android-only implementation
❌	Product README	Repo README is stock Flutter boilerplate

EVIDENCE

`build.gradle.kts:8-41` · `AndroidManifest.xml (no service/receiver)` · `README.md (boilerplate)` · `test/`

17 WHAT ACTUALLY GETS INSTALLED

Deployment Architecture



The entire product is one Android APK on the user's phone, talking to a BLE band and — only in an emergency — to a contact over the cellular network. There is no server tier.

Build facts: `compileSdk 37` (pinned above Flutter's default because the secure-storage lib requires it), Java/Kotlin **17**, `minSdk`/`targetSdk`/`versionCode`/`versionName` inherited from Flutter defaults / `pubspec.yaml` (`1.0.0+1`). Root Gradle only redirects the build dir; repos are `google()` + `mavenCentral()`.

EVIDENCE

`android/app/build.gradle.kts` · `android/build.gradle.kts` · `MainActivity.kt` · `AndroidManifest.xml`

18 ROADS NOT TAKEN

Alternatives & Why Not

Decision	Alternative considered	Why the current choice
On-device inference	Cloud ML API	Must work offline (disaster/rural); privacy; zero latency & cost. Cloud would break the core promise.
Heuristics + on-device learning for risk	A single supervised trained model	No labelled clinical dataset available; heuristics are explainable and safe, and the shipped TFLite model wasn't reliable enough to wire in.
SharedPreferences + sembast	Encrypted SQLCipher via drift	Simpler, pure-Dart, fewer moving parts for a prototype; the encrypted stack is scaffolded for later rather than blocking the demo.
Own native MethodChannels	Third-party Flutter plugins (telephony, geolocator, share_plus)	Avoids AGP-9 / Gradle fragility that broke plugin builds; keeps the native surface small and controlled.
Single ChangeNotifier controller	BLoC / Riverpod / Redux	Fastest to reason about for a prototype; one source of truth. (Trade-off: the controller is large — §25.)
Flutter (Android)	Native Kotlin, or React Native	One codebase, fast iteration, rich charting; native only where unavoidable (emergency/files).

EVIDENCE

pubspec.yaml comments (pure-Dart rationale) · prana_health_controller.dart:170–174 (TFLite replaced) · MainActivity.kt (own channels)

19 WHY IT'S BUILT THIS WAY

Technical Decision Log

1. **Offline-first, on-device.** The product exists for moments with no connectivity; privacy is a bonus. Everything else follows from this.
2. **Transparent risk over black-box.** Physiology-based formulas can be explained and defended; important for a health-adjacent tool without clinical validation.
3. **Real per-person learning.** Running-stats baseline + on-device autoencoder so "abnormal" means "abnormal for you," not a generic threshold.
4. **Uncapped 0–100 risk.** An earlier demo cap was removed so the score reflects the true computed severity, not a flattering ceiling.
5. **Frozen feature schema (v1.1.0).** Guarantees reproducibility and lets the CSV export be de-normalized back to real units exactly.
6. **Own native channels, no new plugins.** Deliberate response to build-toolchain fragility; keeps the native footprint auditable.
7. **Two simple stores now, encryption later.** Ship a working prototype; leave the encrypted-DB scaffolding in place as a clearly-marked next step.

EVIDENCE

app_constants.dart · feature_schema.dart (v1.1.0) · risk_engine.dart · project memory (risk-score-uncapped, no-new-native-plugins)

20 WHERE TO FIND THINGS

File-by-File Codebase Map

The app lives under `lib/`, organized as `core/` (cross-cutting), `features/` (one folder per screen/domain), and `shared/` (widgets/models). Highest-value files:

File	What it does
lib/main.dart	Bootstrap: init storage, build the ML pipeline, inject it, run the app
lib/app/app.dart	MaterialApp, splash→shell routing, the 5-tab navigation shell, header SOS wiring
lib/app/routes.dart	Named routes for off-shell screens
lib/core/services/prana_health_controller.dart	**The brain** — owns services, runs the per-reading pipeline, persists summaries
lib/core/constants/app_constants.dart	All thresholds/priors (HR/SpO_2/temp bounds, fall g-forces, retention days)
lib/core/services/local_storage_service.dart	SharedPreferences persistence (profile, baseline, settings, daily summaries)
lib/features/ml/feature_schema.dart	Frozen v1.1.0 definition of the 16-feature vector

lib/features/ml/feature_processor.dart	Builds the 16-feature vector from readings
lib/features/ml/mock_inference_service.dart	The 5 hand-tuned physiological risk formulas
lib/features/ml/personalized_inference_service.dart	**The live inference engine** — heuristics + personalization + safety floors
lib/features/ml/personal_model.dart	Welford running stats + Mahalanobis anomaly
lib/features/ml/autoencoder.dart	On-device-trained autoencoder (16-8-4-8-16)
lib/features/ml/vitals_sample_store.dart	sembast store for calm samples + persisted model
lib/features/ml/tflite_inference_service.dart	**Dead code** — TFLite loader, never instantiated
lib/features/risk/risk_engine.dart	Combines inference into the 0–100 score + drivers; calls the recommender
lib/features/recommendations/recommendation_engine.dart	Rule cascade → specific advice
lib/features/devices/*sensor_data_source.dart	BLE source, mock (demo) source, and the shared interface
lib/features/devices/sensor_parser.dart	Parses BLE packets (custom JSON + standard GATT) into typed readings
lib/features/emergency/*	Fall detection, emergency state machine, native dispatcher, SOS button
lib/features/report/*	PDF report data + pure-Dart PDF builder + share service
lib/features/data_export/*	CSV export + feature-vector decoder (inverse of the processor)
android/.../MainActivity.kt	Native Kotlin: the two MethodChannels (emergency, files)

21

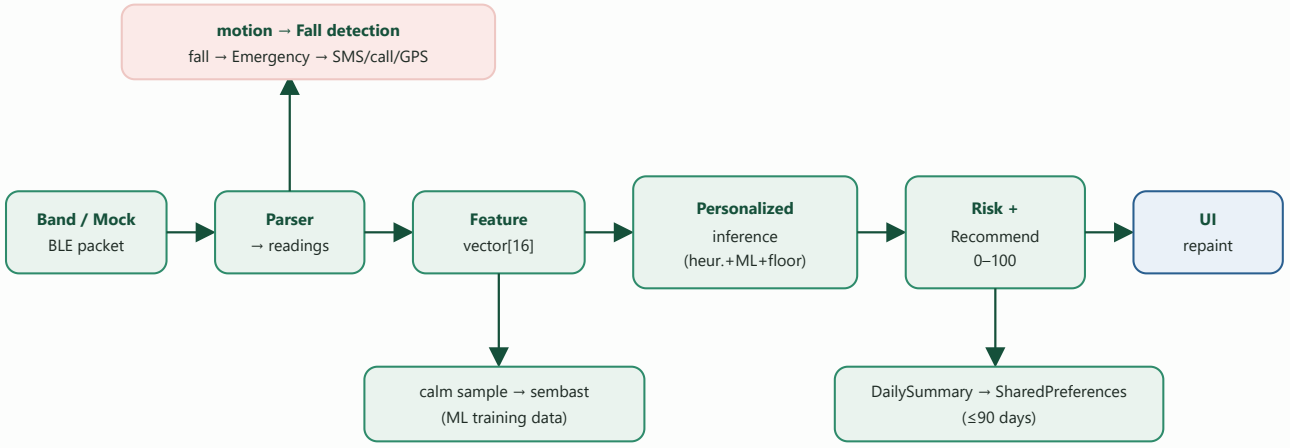
THE FUNCTIONS THAT MATTER

Important Functions Explained

Function	What it does (plain → precise)
`main()` `main.dart`	Starts the app. Initializes storage, opens the vitals DB, loads or creates the personal model, wraps the heuristic service in the personalized one, and launches the widget tree with three providers.
controller reading pipeline `prana_health_controller.dart`	For each incoming reading: parse → update baseline → extract features → infer → score risk → recommend → `notifyListeners()` → persist. The single place data becomes insight.
`extractFeatureVector()` `feature_processor.dart`	Turns raw readings + baseline into the fixed 16-number vector the models consume; guaranteed finite and length-16 (unit-tested).
`predict()` `mock_inference_service.dart`	Applies five sigmoid formulas to the vector → heat/respiratory/cardiovascular/fatigue/overall probabilities. Pure, deterministic, explainable.
`predict()` (personalized) `personalized_inference_service.dart`	Calls the heuristic `predict()`, then replaces/augments the overall anomaly using the Mahalanobis distance + autoencoder reconstruction error, clamped by physiological safety floors.
`update()` / distance `personal_model.dart`	Welford's online algorithm updates per-feature mean/variance from calm readings; Mahalanobis distance scores how far a new reading is from your normal.
`train()` / `reconstruct()` `autoencoder.dart`	Trains a 16-8-4-8-16 net by back-prop in a background isolate; reconstruction error on new input is the "unusual for you" signal.
`calculateRisk()` `risk_engine.dart`	Fuses domain probabilities + anomaly into the uncapped 0–100 score and level, attaches named drivers, and asks the recommender for advice.

`generateRecommendations()` `recommendation_engine.dart`	Ordered if/else over five scenarios (heat, respiratory, cardiovascular, fatigue, mixed) → primary + secondary advice with live values interpolated.
fall detection `fall_detection_service.dart`	3-stage rule on the band's IMU: explicit fall flag, or >2.8 g impact followed by <0.25 g stillness within ~2 s → arms the emergency countdown.
emergency dispatch `emergency_dispatcher.dart`	Over `prana/emergency`: gets GPS, sends an SMS (identity + vitals + Maps link) and places a call; degrades to clipboard if blocked.
`buildHealthReportPdf()` `report_pdf_builder.dart`	Pure function → PDF bytes: patient card, baseline, period KPIs, hand-drawn trend charts, readings table, disclaimer.
`buildCsv()` `data_export_service.dart`	RFC-4180 CSV of decoded calm readings + the raw feature vector (self-verifying export).

22 FOLLOW ONE READING
Data Flow



One reading, end to end: the green path is per-tick inference; blue is the UI; the lower branches persist data; the red branch is the independent fall→emergency path.

EVIDENCE

`prana_health_controller.dart` · `sensor_parser.dart` · `feature_processor.dart` · `personalized_inference_service.dart` · `risk_engine.dart` · `vitals_sample_store.dart` · `local_storage_service.dart` · `fall_detection_service.dart`

23 OUTSIDE CONNECTIONS
APIs & Third-Party Services

NO THIRD-PARTY OR CLOUD SERVICES

No analytics, no crash reporting, no maps SDK, no cloud storage, no auth provider, no AI API. Not found in the current codebase.

The only external interfaces are **platform** APIs on the phone itself and the **Bluetooth** link:

Interface	Used for
Android `SmsManager`	Sending the emergency SMS
Android telephony (`ACTION_CALL`)	Placing the emergency call
Android `LocationManager`	One-shot GPS fix for the alert
androidx `FileProvider`	Sharing/opening the generated PDF & CSV
BLE GATT (flutter_blue_plus)	Reading the wearable's characteristics

EVIDENCE

`MainActivity.kt` · `emergency_dispatcher.dart` · `ble_constants.dart` · `bluetooth_sensor_data_source.dart`

24 WHAT'S PROVEN
Testing

IN SIMPLE TERMS

The trickiest core logic has automated tests — the parts where a bug would be dangerous or invisible (baseline maths, feature extraction, risk levels, packet parsing, the emergency state machine). The UI and some newer pieces are not yet covered by automated tests.

Test file	Cases	Covers
baseline_service_test.dart	4	Default bounds, HR deviation %, SpO_2 drop, learning without error
emergency_service_test.dart	3	State machine: ready → SOS sent (honest "location unavailable") → resolved
feature_processor_test.dart	1	Exactly 16 finite features
mock_sensor_data_source_test.dart	2	Simulated fall emits; scenario switching doesn't error
risk_engine_test.dart	2	Safe under rest; high/critical under extreme heat + tachycardia
sensor_parser_test.dart	6	Valid/invalid JSON; 8- & 16-bit GATT HR, SpO_2, battery

Total: 6 files, ~18 unit tests (pure Dart, `flutter_test`). **Not covered:** PDF builder, CSV export, recommendation engine, ML personalization, and any widget/integration test.

EVIDENCE

`test/ (six files)` · `analysis_options.yaml (flutter_lints, no extra rules)`

25 OWN THESE BEFORE A JUDGE FINDS THEM
Current Limitations

Limitation	Honest framing

Shipped TFLite model unused	Live path is the personalized service; the `.tflite` is trained but not wired in
No accuracy metrics	No precision/recall/AUC; on-screen confidence (~0.92) is a placeholder constant
No encryption at rest	Plaintext SharedPreferences + sembast; SQLCipher scaffolded but inert
Debug signing on release	No production keystore configured yet
Locked-screen fall reliability	No foreground/background service; detection is best-effort while the app is alive
Android-only native features	Emergency/file channels not implemented for iOS/desktop
Sleep data is sample data	Sleep staging is demo-only, gated behind demo mode; no sensor produces it
Single-user, no sync	No accounts, no cross-device or cross-user aggregation
Large controller	The central controller is a ~700-line "god object" — a known refactor target
Some 2-sample trends	Certain trend readouts use a short delta rather than a long window
App ID / README	Still the template `com.example` ID; README is stock boilerplate
Depends on a compatible band	Real vitals need a band speaking the expected BLE contract; else demo mode

THE MOVE

Volunteer two or three of these before you're asked. Judges trust a team that clearly knows its own edges far more than one that claims perfection.

26 WHERE IT GOES NEXT
Future Scope

Short term

- Real release signing + proper app ID
- Turn on encryption at rest (the scaffolding is already there)
- Foreground service for reliable locked-screen fall alerts
- Product README + onboarding polish

Medium term

- Collect labelled data; train & wire a real supervised model with published metrics
- iOS native emergency/file channels
- Widget/integration test suite
- Optional clinician-shareable dashboard

Long term

- Opt-in encrypted cloud sync & multi-device
- Population/fleet view for authorities during disasters (privacy-preserving)
- Federated learning so the model improves across devices without sharing raw data
- Richer sensing (ECG, cuff BP) and clinical validation toward regulatory standards

FRAMING

Notice the theme: most near-term items are **finishing things already scaffolded** (encryption, signing), which is a credible, low-risk roadmap — not a wish list.

27 FACE THE JUDGES

Judge Question Prep — 40+

Each question has three lines: Say (your out-loud answer), Tech (the precise backup if pressed), and Proof (the file that settles it). Read the Say lines end-to-end the night before; the rest is your safety net.

A · PRODUCT & PROBLEM

1. What problem does PRANA solve?

Say: It's a personal early-warning system for your health and your surroundings that keeps working with no internet — for heatwaves, bad air, and emergencies where help is far away.

Tech: Offline-first monitoring of body vitals + environment, a personalized risk score, and autonomous emergency dispatch.

Proof: `whole app is offline; emergency_dispatcher.dart`

2. Who is it for?

Say: Vulnerable people and outdoor/rural workers — the elderly, those with heart or respiratory conditions, anyone exposed to heat or pollution where a hospital isn't next door.

Tech: Profile carries a "vulnerable" flag and conditions that feed risk framing; scenarios model heatwave, pollution, flood, cyclone, elderly.

Proof: `profile model · mock_sensor_data_source.dart (scenarios)`

3. Why does offline matter so much?

Say: The moments this app is built for — disasters, remote areas — are exactly when networks fail. If it needed the cloud, it would fail when you need it most.

Tech: No network client for app data; all inference/storage on device; only emergency uses cellular SMS/call.

Proof: `§13; no HTTP client in repo`

4. What makes it different from a smartwatch app?

Say: Two things: it learns *your* normal on the device and explains its reasoning, and it can raise a real alarm by itself — SMS, call and location — with no phone-maker cloud in the loop.

Tech: On-device personalization (Welford + autoencoder) + transparent heuristics + native autonomous dispatch.

Proof: `personal_model.dart · autoencoder.dart · MainActivity.kt`

5. Is this a medical device?

Say: No — it's a wellness and early-warning aid, and the report literally says it's not a diagnostic device. That honesty is deliberate.

Tech: Disclaimer embedded in the generated PDF; no diagnostic claims anywhere.

Proof: `report_pdf_builder.dart (disclaimer ~:1026–1047)`

B · ARCHITECTURE & TECH

6. What's the tech stack?

Say: Flutter and Dart for the app, Bluetooth to the band, two small on-device stores, and a little native Kotlin for emergency and file sharing.

Tech: Flutter 3.47.2 / Dart 3.13.2, Material 3, Provider; flutter_blue_plus, shared_preferences, sembast, pdf; Kotlin MethodChannels.

Proof: `pubspec.yaml · pubspec.lock`

7. How is state managed?

Say: One central "brain" object holds all the live data and tells the screens when to refresh.

Tech: Provider + a single `ChangeNotifier` (`PranaHealthController`) as the source of truth; two `ValueNotifiers` for high-frequency updates.

Proof: `prana_health_controller.dart · main.dart:39–44`

8. Why one big controller instead of BLoC/Riverpod?

Say: For a prototype, one source of truth is the fastest to build and reason about. I know it's grown large — splitting it is on the roadmap.

Tech: ~700-line controller; conscious trade-off of simplicity vs. modularity.

Proof: `prana_health_controller.dart (§25)`

9. Why write your own native code instead of plugins?

Say: The third-party plugins we tried broke the Android build toolchain, so we wrote a thin, controlled native layer ourselves — just two channels.

Tech: AGP-9/Gradle fragility; own `:app` MethodChannels keep the native surface small and auditable.

Proof: `MainActivity.kt · pubspec.yaml comments`

10. Walk me through what happens on one reading.

Say: Parse the packet, update your baseline, build a 16-number feature vector, run inference, score risk, pick advice, refresh the screen, and save the day's summary.

Tech: The controller pipeline; see §4/§22.

Proof: `prana_health_controller.dart (notifyListeners ~:458)`

11. Is it cross-platform?

Say: The Flutter UI is portable, but the emergency and file features are implemented for Android today; iOS is a clear next step.

Tech: Native channels implemented only in the Android `MainActivity`.

Proof: `MainActivity.kt (Android only)`

C · ML / AI (THE HARD ONES)

12. Is there real machine learning, or just if-statements?

Say: Both, and I keep them separate. The *personalization* is real on-device learning — running statistics plus a small autoencoder trained on the phone. The specific risk scores are transparent physiology-based formulas, on purpose.

Tech: PersonalModel (Welford + Mahalanobis) + on-device autoencoder = learning; MockInferenceService sigmoids = heuristics.

Proof: `personal_model.dart · autoencoder.dart · mock_inference_service.dart`

13. Where is the model actually trained?

Say: On the device itself, from your own calm readings — no server, no pre-shipped weights for personalization.

Tech: Autoencoder trained by SGD/back-prop in a `compute()` isolate; weights persisted to sembast and reloaded.

Proof: `autoencoder.dart:18–405 · vitals_sample_store.dart`

14. You ship .tflite files — is that your model?

Say: Honest answer: no. One is a real model we trained offline but haven't wired into the live path; the other is a placeholder. The live intelligence is the personalized service.

Tech: `TFLiteInferenceService` is never instantiated; `prana_risk_model.tflite` is 92 bytes of ASCII.

Proof: `tflite_inference_service.dart (dead) · assets/models/`

15. What's the accuracy?

Say: We don't quote an accuracy number because we haven't measured one on labelled clinical data — I won't claim a figure I can't back. The confidence you see on screen is a placeholder.

Tech: No precision/recall/AUC anywhere; confidence ≈0.92 is a constant.

Proof: `generate_sih_qa.dart admission · code review`

16. What are the 16 features?

Say: Your vitals, how far they deviate from your personal normal, a heart-rate trend, the environment around you, and your activity level.

Tech: Frozen schema v1.1.0: absolute HR/SpO_2/temp, baseline deviations, OLS HR slope, heat index/AQI/humidity, categorical activity.

Proof: `feature_schema.dart · feature_processor.dart · hr_trend_calculator.dart`

17. How does it know what's normal for me?

Say: It quietly averages your calm readings over time and measures how far a new reading strays from that.

Tech: Welford online mean/variance per feature; Mahalanobis distance for multivariate deviation.

Proof: `personal_model.dart:41–68`

18. Couldn't it "learn" a dangerous reading as normal?

Say: No — we put hard physiological floors under the score, so a genuinely dangerous vital always registers, no matter what the personal model learned.

Tech: Safety clamps in the personalized service; absolute thresholds in `app_constants.dart`.

Proof: `personalized_inference_service.dart:71–139 · app_constants.dart`

19. Why heuristics for risk instead of a trained classifier?

Say: Because I can explain and defend every term to a clinician, and because we don't have a labelled dataset to train a trustworthy classifier yet. Explainability beats a black box for a safety tool.

Tech: Sigmoids over the feature vector with coefficients from published physiological thresholds.

Proof: `mock_inference_service.dart:51–96`

D · DATA & PRIVACY

20. Where does my data go?

Say: Nowhere. It stays on your phone. There's no server and no account.

Tech: SharedPreferences + sembast on-device; no upload path exists.

Proof: `local_storage_service.dart · vitals_sample_store.dart`

21. Is the stored data encrypted?

Say: Not yet — I'll be straight about that. The encryption components are in place but not switched on; it's our top security task. Our current protection is that nothing ever leaves the device.

Tech: Plaintext stores; SQLCipher + secure-storage scaffolded but the key manager is never called.

Proof: `database_key_manager.dart (inert) · §11`

22. Can a user delete their data?

Say: Yes — the privacy screen shows exactly what's stored and lets you export or erase it.

Tech: Privacy service reports real counts and performs export/erase.

Proof: `privacy_service.dart · privacy_screen.dart`

23. What permissions do you request, and why?

Say: Bluetooth and location for the band, and SMS + phone-call for emergencies — each tied to a feature, nothing extra.

Tech: BLUETOOTH_*, ACCESS_*_LOCATION, SEND_SMS, CALL_PHONE.

Proof: `AndroidManifest.xml`

24. The emergency SMS contains health data — isn't that a privacy risk?

Say: It's a deliberate trade-off: in an emergency, giving a trusted contact your vitals and location saves time. It only goes to the contact *you* saved, and only when triggered.

Tech: SMS is inherently unencrypted; sent only to the user-configured contact on SOS/fall.

Proof: `emergency_dispatcher.dart`

E · EMERGENCY & SENSORS

25. Does the SOS actually send anything, or is it a mock-up?

Say: It's real. It sends an SMS with your identity, vitals and a Google-Maps location link, and it places a phone call — through Android's own SMS and calling APIs.

Tech: `prana/emergency` channel → `SmsManager.sendSms`, `ACTION_CALL`, `LocationManager`.

Proof: `MainActivity.kt · emergency_dispatcher.dart`

26. How does fall detection work?

Say: The band's motion sensor sees a sharp impact followed by stillness; that pattern triggers a 10-second countdown you can cancel before it alerts anyone.

Tech: 3-stage rule: explicit flag OR >2.8 g impact then <0.25 g stillness within ~2 s; then cancelable countdown.

Proof: `fall_detection_service.dart:13–39 · app_constants.dart`

27. What stops false alarms?

Say: Two things: the impact-then-stillness pattern (not just a bump), and a countdown that lets you cancel before anything is sent.

Tech: Multi-stage gate + `sosCountdownSeconds = 10` cancel window.

Proof: `fall_detection_service.dart · emergency_screen.dart`

28. Careful — could a test fall in the demo send a real SMS?

Say: Yes, and I'm glad you asked — dispatch is real and *not* gated by demo mode. For the demo I've cleared the emergency contact (or I'll cancel during the countdown) so we don't text a real person.

Tech: Emergency fires on the state transition regardless of mock mode; guard by clearing the contact.

Proof: `emergency_service.dart · emergency_dispatcher.dart`

29. Where do the vitals come from — is the data faked?

Say: Real vitals come from the band over Bluetooth. There's also a clearly-labelled Demo mode with realistic scenarios for when no band is present.

Tech: Default source is BLE; `MockSensorDataSource` is opt-in and physiologically smoothed.

Proof: `bluetooth_sensor_data_source.dart · mock_sensor_data_source.dart`

30. What if the band disconnects?

Say: The screens show honest empty-states and prompt to reconnect — it never invents numbers.

Tech: `hasLiveData` gates the UI; parser emits only present fields.

Proof: `environment_screen.dart:65 · sensor_parser.dart`

F · DEPLOYMENT & PRODUCTION

31. Is it ready for the Play Store?

Say: It's a solid prototype. Before store release it needs real signing, a proper app ID, and encryption switched on — all known, scoped items.

Tech: Debug signing; template app ID; encryption inert. (§16)

Proof: `build.gradle.kts:37–41`

32. How is it built and run?

Say: One command builds a single APK; it runs on a normal Android phone with no server to stand up.

Tech: `compileSdk 37`, Java/Kotlin 17, Flutter default min/target SDK; `flutter run`.

Proof: `android/app/build.gradle.kts`

33. Will fall detection work with the screen off?

Say: Best-effort today — there's no background service yet, so reliability with the app killed isn't guaranteed. Adding a foreground service is the fix, and it's on the roadmap.

Tech: No foreground/background service or receiver declared.

Proof: `AndroidManifest.xml (§25/§26)`

34. How do you get data to a doctor?

Say: One tap generates a clean multi-page PDF with charts and a summary, shareable like any file. You can also export the raw readings as CSV.

Tech: Pure-Dart `pdf` builder; share via the `prana/files` FileProvider channel.

Proof: `report_pdf_builder.dart · data_export_service.dart`

35. What does it cost to run at scale?

Say: Almost nothing on our side — there are no servers. Every phone does its own work, so a million users cost us no infrastructure.

Tech: Fully on-device; no backend to scale. (§15)

Proof: `no server tier exists`

G · HONESTY & LIMITATIONS

36. What's the weakest part of the app?

Say: Two: no encryption at rest yet, and no measured accuracy for the models. Both are known, and both have a clear plan.

Tech: Plaintext stores; no metrics. (§25)

Proof: `§11 · §9`

37. What would you fix first with more time?

Say: Turn on encryption (the scaffolding's already there), add a foreground service for reliable fall alerts, and set up real release signing.

Tech: Finishing scaffolded work, not new architecture.

Proof: `§26`

38. How much did you build vs. libraries?

Say: The intelligence and logic are ours — the feature pipeline, the personalization, the risk and recommendation engines, the native emergency layer. Libraries handle plumbing like BLE and PDF.

Tech: Custom: ML, risk, recommendations, parser, native channels. Libraries: transport/format only.

Proof: `lib/features/* · MainActivity.kt`

39. Is any of the demo faked for the pitch?

Say: The disaster scenarios and the sleep panel are clearly demo data — I'll label them as such live. Everything else is the real pipeline.

Tech: Scenarios in mock source; sleep is a fixed literal behind demo mode.

Proof: `mock_sensor_data_source.dart · controller (sleep sample)`

40. What's genuinely innovative here?

Say: On-device personalized learning that works fully offline, combined with autonomous, real emergency dispatch — most consumer apps have one or the other, and route through the cloud. We do both, on the phone.

Tech: Edge learning (Welford + autoencoder) + native SMS/call/GPS with graceful degradation.

Proof: `autoencoder.dart · personal_model.dart · MainActivity.kt`

41. Why should this win?

Say: Because it solves a real problem for the people most exposed, it works when the network doesn't, it's honest about what it can and can't do, and every claim we make is backed by code we can open right now.

Tech: Offline-first, explainable, on-device ML, real emergency response — all verifiable in this repo.

Proof: `this entire document`

28

FACE THE JUDGES

Top 20 High-Risk Questions

The ones that can rattle you — because the honest answer is "not yet" or "it's a demo." Memorise the one-liner. The move every time is the same: admit it plainly, then show the plan. Judges reward the candour; they punish the bluff.

#	The scary question	Your one-line answer
1	"Is the ML real or just if-statements?"	Personalization is real on-device learning; the risk scores are transparent physiology formulas — and I keep the two clearly separate.
2	"What's your accuracy?"	We don't quote one — we've not measured against labelled clinical data, and I won't claim a number I can't defend.
3	"You ship .tflite files — is that the model?"	No. One's a real model not yet wired in, one's a placeholder; the live intelligence is the personalized Dart service.
4	"Is the data encrypted at rest?"	Not yet — the components are scaffolded but off. Our protection today is that nothing ever leaves the phone. It's my #1 fix.
5	"Could a test fall text a real person?"	Yes — dispatch is real and not demo-gated. I've cleared the contact for the demo, or I'll cancel in the 10-second window.
6	"Is this production-ready?"	It's a strong prototype; release needs real signing, a proper app ID, and encryption on — all known and scoped.
7	"What's the 0.92 confidence?"	A placeholder constant, not a measured metric — I'd rather tell you that than pretend.
8	"Does fall detection work with the app closed?"	Best-effort today; there's no background service yet. Adding a foreground service is the fix and it's on the roadmap.
9	"Is any demo data faked?"	The disaster scenarios and the sleep panel are clearly demo data — I label them live. The pipeline behind them is real.
10	"Could it learn a dangerous reading as normal?"	No — hard physiological floors sit under the personal model, so a genuinely dangerous vital always registers.
11	"Why no trained classifier for risk?"	No labelled dataset yet, and explainability matters for a safety tool — I can defend every term to a clinician.
12	"Isn't one giant controller bad design?"	For a prototype it's the clearest single source of truth; modularising it is a known refactor, not a rewrite.

13	"SMS is unencrypted — privacy concern?"	A deliberate trade-off: in an emergency, speed to a contact you chose beats secrecy. It only fires on trigger.
14	"What if there's no GPS fix?"	The alert still sends with vitals and identity; the location line is simply omitted rather than faked.
15	"How is this different from Google Fit / a watch?"	On-device personalization *and* autonomous real emergency dispatch, fully offline — not routed through a vendor cloud.
16	"What did you actually build vs. libraries?"	The ML, risk, recommendations, parser and native emergency layer are ours; libraries only do BLE transport and PDF format.
17	"iOS?"	The Flutter UI is portable; emergency/file features are Android-native today — iOS is a clear next step.
18	"Where's the trained neural network at runtime?"	The autoencoder — trained on-device from your calm readings — runs for personalization; it is not a shipped pre-trained net.
19	"Can you prove these claims?"	Yes — every section here cites the exact file. I can open any of them right now.
20	"Why should this win?"	Real problem, works offline when it matters most, honest about limits, and every claim is backed by code in this repo.

THE ONE RULE

Never bluff a metric or a capability. A confident "not yet, and here's the plan" wins the room; a fabricated number loses it the moment someone asks a follow-up.

29

FACE THE JUDGES

The 60-Second Explanation

Learn this almost word-for-word. It's your opener when a judge says "so, what is it?" — one calm minute, no jargon.

"PRANA is a personal early-warning system for your health and your surroundings — and it works with no internet at all.

You wear a small band; it streams your heart rate, oxygen, temperature and the air and heat around you to the phone over Bluetooth. On the phone — not on any server — PRANA learns what *normal* looks like for *you*, and flags when something drifts toward dangerous. It turns that into a single risk score with a plain-language reason, so you actually understand it.

And if something goes badly wrong — a fall, or you hit SOS — it acts on its own: it texts a trusted contact your vitals and a map link, and places a call. No cloud, no account, nothing leaves your phone unless it's an emergency.

We built it for the people most exposed — the elderly, outdoor workers, anyone far from a hospital — because those are exactly the moments the network isn't there. It's honest about being a wellness aid, not a medical device — and every feature I just described is real code we can open right now."

DELIVERY TIP

Pause after "with no internet at all" and after "acts on its own." Those are the two lines judges remember.

30 FACE THE JUDGES
3-Minute Technical Presentation

For the technical round. Six beats, ~30 seconds each. Say the beat, don't read it — the bold phrase is your anchor.

Time	Beat	What you say
0:00–0:30	Problem & stance	"PRANA is an **offline-first, on-device** health-and-environment early-warning app. No backend, no cloud, no account — everything runs on the phone, which is the whole point for disasters and remote areas."
0:30–1:00	Data in	"A BLE band streams vitals and environment. We parse the packet, then build a **frozen 16-feature vector** — absolute vitals, how far each strays from your personal baseline, a heart-rate trend, and the environment." `feature_processor.dart`
1:00–1:40	The intelligence	"Two layers, kept separate. **Real on-device learning** — Welford running statistics plus a small autoencoder we train on the phone from your calm readings — personalizes to you. Then **transparent physiology-based formulas** turn the vector into risk scores I can defend term by term." `personal_model.dart · autoencoder.dart · mock_inference_service.dart`
1:40–2:10	Safety net	"Because personalization could drift, we clamp with **hard physiological floors** — a dangerous vital always registers. Fall detection is a multi-stage rule with a cancelable countdown to kill false alarms." `personalized_inference_service.dart · fall_detection_service.dart`
2:10–2:40	Autonomous response	"Emergencies are **real native dispatch** — a Kotlin channel calls Android's SmsManager, ACTION_CALL and LocationManager to text vitals + a map link and place a call. It degrades gracefully if GPS or a contact is missing." `MainActivity.kt · emergency_dispatcher.dart`
2:40–3:00	Honest close	"Storage is on-device in two stores; encryption is scaffolded and next on our list. We quote **no accuracy number** because we haven't measured one — and we'd rather be trusted than impressive. Every claim maps to a file in this repo."

IF THEY STOP YOU EARLY

Make sure you've said beats 3 and 5 (the intelligence and the emergency dispatch) — those are the two that carry the score. Everything else is supporting detail.

31 FACE THE JUDGES
Demo Day Flow

The exact click-path, in an order that tells a story and never surprises you. Rehearse it twice before you present.

Before you walk up (the safety checklist)

DO THIS FIRST — EVERY TIME

Clear the emergency contact in Profile (or be ready to cancel the 10-second countdown). Emergency dispatch is *real* and is *not* disabled in demo mode — a live fall test **will** text and call a real number otherwise.

✓	Check

☐	Phone charged, screen-timeout long, Do-Not-Disturb off (so alerts are visible).
☐	Band paired and streaming *or* Demo mode ready — decide which you're showing.
☐	Emergency contact cleared (see red box above).
☐	A report already generated once, so the PDF share sheet is warm.

The 5-stop tour

#	Screen	What you do & say
1	Dashboard (Home)	Open on the risk score. "This one number is your current risk, in plain words — worked out entirely on this phone." Point at the greeting + score card.
2	Insights (center button)	Tap the raised center button. "Here's *why* — the drivers behind the score, and how it's personalized to me." Show the explanation.
3	Environment	"And the world around me — feels-like temperature, air quality, safe-exposure time." Show the real cards (AQI now reflects the live reading).
4	SOS (header disc)	Tap the SOS disc in the header. "In an emergency this fires on its own — SMS with my vitals and location, plus a call." **Cancel the countdown** to prove the false-alarm guard.
5	Report / Privacy	Generate the PDF. "A doctor-ready summary in one tap." Then Privacy: "and I can see exactly what's stored and wipe it — nothing ever left the phone."

IF THE BAND DROPS MID-DEMO

Don't panic — switch to Demo mode and say so out loud: "I'll show this on our labelled demo data." The honesty reads as confidence, and the screens show honest empty-states either way.

32 FACE THE JUDGES
Explain Like I'm 10

When someone non-technical asks — a family member, a general-track judge — reach for the story, not the spec.

> **"Imagine a really attentive friend who sits on your wrist.**
>
> This friend quietly learns what your normal day feels like — how fast your heart usually beats, how warm you usually are. After a while they know *your* normal, not some average stranger's.
>
> So when something's off — you're getting too hot, or your heart is racing more than it should — they lean over and say, gently, 'hey, something's not right, here's why.'
>
> And if you fall and can't get up, this friend doesn't wait. They pull out a phone, text your family exactly where you are and how you're doing, and call for help — all by themselves.
>
> The best part: this friend never gossips. Everything they notice stays with you, on your phone. They don't phone home to a company, and they don't need the internet to look out for you."

WHY THIS LANDS

It maps every real feature to a human idea: personalization → "learns your normal," explainable risk → "tells you why," autonomous dispatch → "calls for help by themselves," on-device privacy → "never gossips." No lie, just plain words.

FACE THE JUDGES

33 One-Page Cheat Sheet

If you read one page before you walk in, read this one.

✅ 10 sentences I MUST remember

1. PRANA is **offline-first** — no backend, no cloud, no account; everything runs on the phone.
2. It **learns my personal normal** on-device with Welford stats and an autoencoder trained on the phone.
3. Risk scores are **transparent physiology formulas** I can defend term by term — not a black box.
4. The two are kept **separate**: real learning for personalization, explainable heuristics for the score.
5. **Hard physiological floors** mean a genuinely dangerous vital always registers.
6. Emergency dispatch is **real**: native SMS + call + GPS via Android APIs, and it degrades gracefully.
7. Fall detection is a **multi-stage rule** with a cancelable 10-second countdown for false alarms.
8. The risk score runs the **full natural 0–100 range** — no artificial cap.
9. All data stays **on the phone**; the user can view, export and erase it.
10. **Every claim maps to a file** — I can open the code right now.

⛔ 10 things I MUST NOT claim

1. ❌ "There's a trained neural network scoring risk in the live path." (Heuristics do that.)
2. ❌ "It's 92% accurate." (0.92 is a **placeholder constant**; no metric is measured.)
3. ❌ "The data is encrypted at rest." (Stores are **plaintext**; encryption is scaffolded, off.)
4. ❌ "The .tflite files power the app." (One's unused, one's a 92-byte placeholder.)
5. ❌ "It's a medical / diagnostic device." (Wellness & early-warning aid only.)
6. ❌ "Fall detection works reliably with the app killed." (No background service yet.)
7. ❌ "It's ready for the Play Store." (Debug signing, template app ID.)
8. ❌ "Demo mode disables emergency dispatch." (It does **not** — dispatch is always real.)
9. ❌ "The disaster scenarios / sleep panel are live sensor data." (Clearly labelled demo data.)
10. ❌ "We validated it on clinical / labelled datasets." (We have not.)

Numbers & names to have on the tip of your tongue

Thing	Value	Where
Feature vector	16 features, frozen schema v1.1.0	`feature_schema.dart`
Autoencoder shape	16-8-4-8-16, trained on-device (SGD)	`autoencoder.dart`
Risk sub-scores	5 (heat, respiratory, cardio, fatigue, overall)	`mock_inference_service.dart`
Fall thresholds	impact >2.8 g → stillness <0.25 g within ~2 s	`app_constants.dart`
SOS cancel window	10 seconds	`emergency_screen.dart`
Risk range	0–100, uncapped	`risk_engine.dart`
Storage	SharedPreferences + sembast (`prana_ml.db`)	on-device, plaintext
Native channels	`prana/emergency`, `prana/files`	`MainActivity.kt`
Stack	Flutter 3.47.2 / Dart 3.13.2, Provider, Material 3	`pubspec.lock`

THE MANTRA

"Offline. On-device. Personalized. Explainable. Honest." Five words — if you blank, start there and the rest comes back.

34 FACE THE JUDGES
Codebase Fact Check

The final honesty pass — every notable claim, graded against the actual source. ✅ Confirmed = fully in code. ⚠️ Partial = present but incomplete or demo-bound. ❌ Not implemented = scaffolded or absent. ❓ Can't verify = needs data/hardware we don't have here.

Claim / capability	Status	Evidence & notes
Fully offline; no backend/cloud/account	✅	No HTTP client for app data anywhere in the repo.
BLE ingest of vitals + environment	✅	`bluetooth_sensor_data_source.dart` · `sensor_parser.dart`
16-feature vector, frozen schema	✅	`feature_schema.dart` · `feature_processor.dart`
On-device personalization (Welford + Mahalanobis)	✅	`personal_model.dart:41-68`
Autoencoder trained on-device (backprop, isolate)	✅	`autoencoder.dart`; persisted to sembast.
Personalized service is the live inference path	✅	`main.dart` injects it via `customMlService`.
Risk scores from trained ML classifier	❌	Scores are heuristic sigmoids — `mock_inference_service.dart`. (By design.)
Bundled .tflite models power the app	❌	`TFLiteInferenceService` never instantiated; one model is a 92-byte placeholder.
Safety floors under personalization	✅	`personalized_inference_service.dart:71-139`
Measured accuracy / precision / recall	❌	No metrics; on-screen confidence ≈0.92 is a constant.
Real emergency SMS + call + location	✅	`MainActivity.kt` → SmsManager / ACTION_CALL / LocationManager.
Emergency degrades gracefully (no GPS/contact)	✅	`emergency_dispatcher.dart` omits missing fields.
Fall detection (multi-stage + countdown)	✅	`fall_detection_service.dart:13-39`; band IMU only.
Fall detection with app closed / screen off	⚠️	No foreground/background service declared — best-effort only.
Encryption at rest	❌	SQLCipher/secure-storage scaffolded; key manager never called.
User can view / export / erase data	✅	`privacy_service.dart` · `privacy_screen.dart`
Doctor-ready PDF + CSV export	✅	`report_pdf_builder.dart` · `data_export_service.dart`
Risk score uncapped 0–100	✅	`risk_engine.dart`; old demo cap removed.
Disaster scenarios are live sensor data	⚠️	Demo-only, physiologically smoothed — `mock_sensor_data_source.dart`.
Sleep panel is measured	⚠️	Fixed literal behind demo mode — not from a sensor.
Release-ready signing / app ID	❌	Debug signing; template `com.example.sih_test_app_new`.

iOS support for emergency/files	❌	Native channels implemented only in Android `MainActivity.kt`.
Cross-platform Flutter UI	✅	UI layer is portable Dart/Flutter.
Real-world clinical validation	❓	Requires labelled clinical data + trials — out of scope for this build.
Battery / long-run performance on real hardware	❓	Not benchmarked here; needs on-device profiling.

BOTTOM LINE

Everything graded ✅ is demonstrable from this repo today. Everything ⚠️/❌ has a named file and a place on the roadmap (§26). Nothing in this manual is a claim you can't open the code to prove — which is exactly what lets you speak to judges with a straight back.

End of manual · **PRANA — App Overview & Judge Prep** · generated 13 September 2026 from the live codebase.

Every claim herein cites the file that proves it. When in doubt, open the code.